WorldGyan
Next-Gen AI & Engineering

STUDY NOTES COLLECTION • VOL 1

Angular Signals

Modern Reactive Architecture Notes

Signal Primitives • Computed Graphs • Effects • Zoneless Change Detection

Prepared by: Ashutosh Kumar Choubey

Part 1 (Reactive Core & Zoneless State)

worldgyan.com • Open Source Study Deck

1 Why Signals Replace Zone.js

Fine-Grained Reactivity: Angular updates only the exact DOM node that depends on the signal, skipping tree traversals.

Synchronous Value Access: Read values directly with mySignal() without subscribing or managing memory leaks.

Zoneless Performance: Eliminates Monkey-Patching from zone.js, saving bundle size and CPU overhead.

2 Signal Declaration & Mutating Patterns

TYPESCRIPT • RUNNABLE

```typescript
import { signal, computed, effect } from '@angular/core';


// 1. Writable signal primitive
export const userCart = signal<{ id: string; qty: number }[]>(
    []);


// 2. Computed derived signal (memoized dependency graph)
export const totalItemCount = computed(() =>
  userCart().reduce((acc, item) => acc + item.qty, 0)
);


// 3. Immutably update state
userCart.update(items => [...items, { id: 'SKU-99', qty: 1
    }]);
```

2. Dynamic Computed Dependency Graphs

1 Glitch-Free Memoization Invariants

Dynamic Branches: Only signals read during the current execution become tracked dependencies.

Glitch-Free Guarantee: Angular guarantees computed values are never evaluated in an inconsistent state.

2 Conditional Branch Dependency Tracking

TYPESCRIPT • RUNNABLE

```
const isVipUser = signal<boolean>(false);
const regularDiscount = signal<number>(0.10);
const vipDiscount = signal<number>(0.25);

// When isVipUser is false, vipDiscount is NOT subscribed!
const appliedDiscount = computed(() => {
  return isVipUser() ? vipDiscount() : regularDiscount();
});
```

Avoid Side Effects in Computed

Computed signals MUST be pure functions. Never perform HTTP calls, mutate external state, or schedule timers inside a computed block!

1 Effect Execution Rules

Injection Context: Effects must be registered inside an injection context (constructor or inject()).

Untracked Dependencies: Use untracked(() => signal()) to read values without adding them as triggers.

Cleanup Handlers: Use onCleanup() callback to cancel active subscriptions, timers, or abort controllers.

2 Effect with Cleanup & Untracked Reads

TYPESCRIPT • RUNNABLE

```typescript
import { effect, untracked } from '@angular/core';


export class LiveTelemetryComponent {

  constructor(private api: TelemetryService) {

    effect((onCleanup) => {

      const activeSession = this.sessionId();

      const currentTenant = untracked(() => this.tenantId());


      const timerId = setInterval(() => {

        this.api.flushHeartbeat(activeSession, currentTenant);

      }, 5000);


      onCleanup(() => clearInterval(timerId));

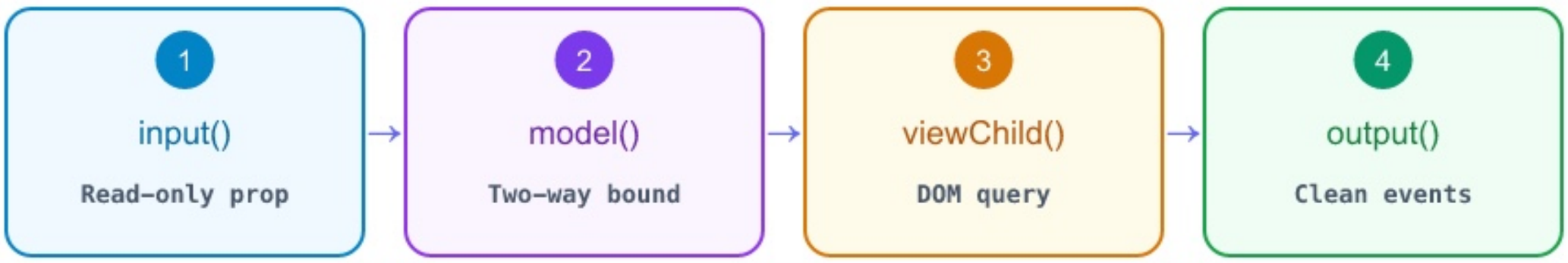
    });

  }

}
```

4. Modern Signal Inputs, Outputs & Queries

1 Component Signal Reactivity Flow



✦ Deterministic Pipeline: Non-Blocking Async Execution • First-Token <180ms • Pydantic Schema Enforcement

2 Clean Declarative Component Architecture

TYPESCRIPT • RUNNABLE

```typescript
import { Component, input, model, output, viewChild,
    ElementRef } from '@angular/core';

@Component({
  selector: 'app-counter-badge',
  template: `<button #btn (click)="increment()">{{ value()
    }}</button>`
})
export class CounterBadgeComponent {
  readonly title = input.required<string>();
  readonly value = model<number>(0);
  readonly changed = output<number>();
  readonly btnRef = viewChild<ElementRef<HTMLButtonElement>>(
    'btn');

  increment(): void {
    this.value.update(v => v + 1);
    this.changed.emit(this.value());
  }
}
```

1 Bridging Asynchronous Streams

toSignal(): Converts RxJS Observables (e.g. HttpClient) to Signals with initial values.

toObservable(): Converts Signal changes into an Observable stream for debounceTime or switchMap.

takeUntilDestroyed(): Automatically unsubscribes when the component lifecycle ends.

2 Live Search with debounceTime & switchMap

TYPESCRIPT • RUNNABLE

```
import { signal } from '@angular/core';
import { toObservable, toSignal } from
    '@angular/core/rxjs-interop';
import { debounceTime, switchMap, distinctUntilChanged } from
    'rxjs/operators';

export class SearchArchitecture {
  readonly searchFilter = signal<string>('');

  // 1. Convert signal to stream for debouncing
  private readonly query$ = toObservable(this.searchFilter).
    pipe(
    debounceTime(300),
    distinctUntilChanged(),
    switchMap(term => this.http.get<Product[]>(
    `/api/search?q=${term}`))
  );

  // 2. Bridge stream back to synchronous signal
  readonly searchResults = toSignal(this.query$, {
    initialValue: [] });
}
```

1 Why Zoneless is the Future

- **Zero Monkey Patching:** Removes 100KB zone.js bundle and microtask queue interception.
- **Targeted DOM Renders:** Scheduler marks only dirty signal nodes instead of entire component trees.
- **Clean Async/Await:** Native async/await works seamlessly with zero Zone context loss.

2 Enabling Zoneless in Modern Angular

TYPESCRIPT • RUNNABLE

```typescript
import { ApplicationConfig,
    provideExperimentalZonelessChangeDetection } from
    '@angular/core';
import { provideRouter } from '@angular/router';


// Modern app.config.ts (Zone.js completely removed from
    angular.json polyfills)
export const appConfig: ApplicationConfig = {
  providers: [
    provideExperimentalZonelessChangeDetection(),
    provideRouter(appRoutes)
  ]
};
```

1 State Architecture Comparison

Paradigm	Legacy Zone.js / RxJS	Modern Signal Standard
• State Primitive	`BehaviorSubject<T> / Observables`	`signal<T>() synchronous reads`
• Derived State	`combineLatest() / map() pipelin…`	`computed(() => a() + b()) memoized`
• Component Inputs	`@Input() decorated properties`	`input<T>() / input.required<T>()`
• Two-Way Binding	`@Input() + @Output() boilerplate`	`model<T>() reactive binding`
• Change Detection	`Dirty-checking Zone tree traver…`	`Fine-grained Zoneless notificatio…`

Signal Architecture Invariant

Signals provide synchronous, glitch-free state for your UI layer. Use RxJS strictly for asynchronous events, debouncing, and web socket coordination!